

ELEC 475 Lab 2, Alistair Barfoot and Luke Barry

Contents

1	Network Architecture	2
2	DataLoader and Dataset	2
3	Hyperparameters	3
4	Augmentation	3
5	Results	3
6	Performance	3
7	Ensemble Approach	3
8	LLM Usage	4

1 Network Architecture

SnoutNet is a Convolutional Neural Network designed for regressing the (x,y) coordinates of an animal's nose from a RGB image. It takes an input image of 227x227x3 and outputs a 2D coordinate vector. It is comprised of three convolutional blocks each followed by a ReLU activation and a Max Pooling layer, then three fully connected (FC) layers.

Layer	Input Shape	Output Size	Parameters
Conv1	227x227x3	114x114x64	in_channels=3, out_channels=64, kernel_size=3, stride=2, padding=1
ReLU1	114x114x64	114x114x64	
MaxPool1	114x114x64	57x57x64	kernel_size=3, stride=2, padding=1
Conv2	57x57x64	29x29x128	in_channels=64, out_channels=128, kernel_size=3, stride=2, padding=1
ReLU2	29x29x128	29x29x128	
MaxPool2	29x29x128	15x15x128	kernel_size=3, stride=2, padding=1
Conv3	15x15x128	8x8x256	in_channels=128, out_channels=256, kernel_size=3, stride=2, padding=1
ReLU3	8x8x256	8x8x256	
MaxPool3	8x8x256	4x4x256	kernel_size=3, stride=2, padding=1
FC1	4x4x256(4096)	1024	
ReLU4	1024	1024	
FC2	1024	1024	
ReLU5	1024	1024	
FC3	1024	2 (u,v)	

2 DataLoader and Dataset

The images and labels are loaded through a custom pytorch dataset class called Custom-Dataset, which reads the annotation file train-noses.txt containing image file names and their corresponding coordinates to their noses. For each entry the dataset loads the image and converts it to an RGB format and parses the coordinate string to numerical values. Images are resized to 227x227 pixels and the transformed nose coordinates are returned as a vector.

A factcheck was performed by creating a small test script that loads a sample and prints the shapes and values. Throughout each train an `example_debug.png` image will appear one in every thousand images with the image and its corresponding snout vector. This ensures that the dataset is loading correctly each time with the augmentation applied.

3 Hyperparameters

The SnoutNet model was trained for 50 epochs with a batch size of 32 using the Adam optimizer. The learning rate scheduler is applied to monitor validation loss and reduces the learning rate by a factor of 2 if no improvement has been observed in 3 consecutive epochs. Early stopping is also implemented if no test loss improvement has been observed for 5 epochs. The learning rate was set to 3×10^{-4} and a weight decay of 1×10^{-4} .

The training was conducted on a NVIDIA RTX 3060 Super laptop edition. Each Epoch took approximately 35 seconds resulting in a max time of approximately 30 minutes for all 50 epochs to be ran.

4 Augmentation

Data augmentation was optionally applied to the training. A horizontal flip to simulate mirrored images was applied as well as a random rotation of $\pm 10^\circ$ to provide rotational variance. For both horizontal flip and a random rotation, the label for the nose vector needed to be adjusted, since the images position within the frame changed the snout of each animal would have changed in position too.

5 Results

Model	Augmentation	Epochs	Train Loss	Test Loss	Time Elapsed (hh:mm)
SnoutNet	None	26	208	487	
	Flip	26	359	532	
	Rotate	33	284	461	
	Flip + Rotate	50	401	419	
SnoutNet-A	None				
	Flip				
	Rotate				
	Flip + Rotate				
SnoutNet-V	None				
	Flip				
	Rotate				
	Flip + Rotate				

6 Performance

7 Ensemble Approach

Model	Augmentation	Localization Error				Localization Error (4 Best)				Localization Error (4 Worst)			
		min	max	mean	stdev	min	max	mean	stdev	min	max	mean	stdev
SnoutNet													
	x												
SnoutNet-A													
	x												
SnoutNet-V													
	x												
SnoutNet-Ensemble													
	x												

Table 1: Localization error statistics for each model and augmentation strategy.

8 LLM Usage

Link to LLM Conversation

For this lab, GitHub Copilot was used to assist in generating the structure of the SnoutNet dataset and DataLoader classes. However, the code generated by the LLM was not just copied and ran in the project. Instead, each line of the code generated by the LLM was carefully reveiwed and tested to ensure that it functioned as intended.

Debugging files and lines were added at every step of the process to ensure that each part was functioning correctly and as predicted. In the final version of the lab, much of the code generated by the LLM has been modified or removed entirely as it was not taken just at face value.